

脑与认知科学大作业：神经网络实现编解码

1 作业要求

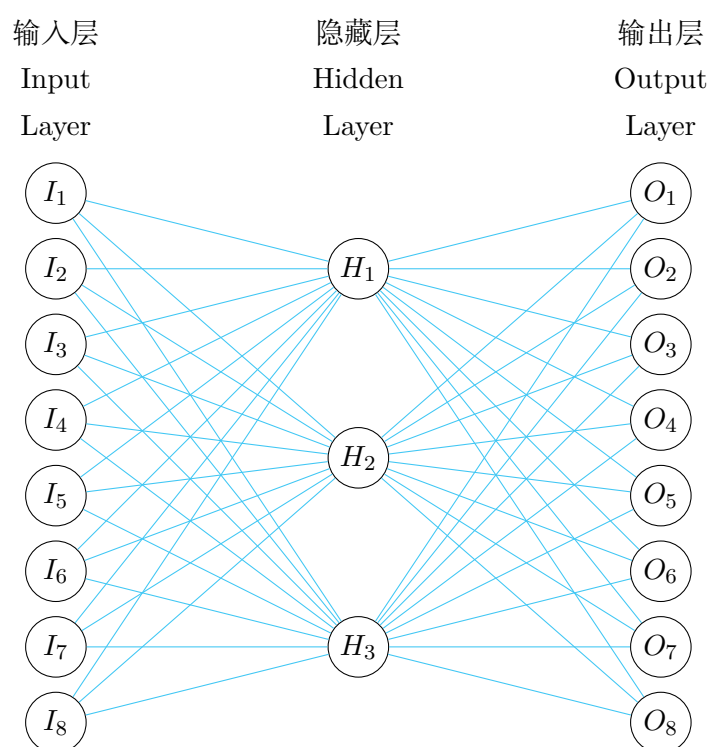


Figure 1: 8-3-8 自编码器神经网络结构图 (标注从 1 到 8)

实现如图 1 的神经网络结构，神经元激活函数使用 Sigmoid 函数。通过神经网络反向传播算法 (Backpropagation Algorithm) 实现神经网络编解码的功能，编解码 8×8 的单位矩阵。编码过程由输入层到隐藏层，解码过程由隐藏层到输出层。手动编写训练算法，逐层实现梯度下降算法。

2 反向传播与梯度下降算法详细推导

反向传播算法 (Backpropagation) 的核心思想是利用微积分中的 ** 链式求导法则 (Chain Rule) **，将输出层的误差反向一层一层地传播给隐藏层，从而计算出网络中每个参数对最终误差的“责任大小” (即偏导数)。然后利用梯度下降算法更新参数。

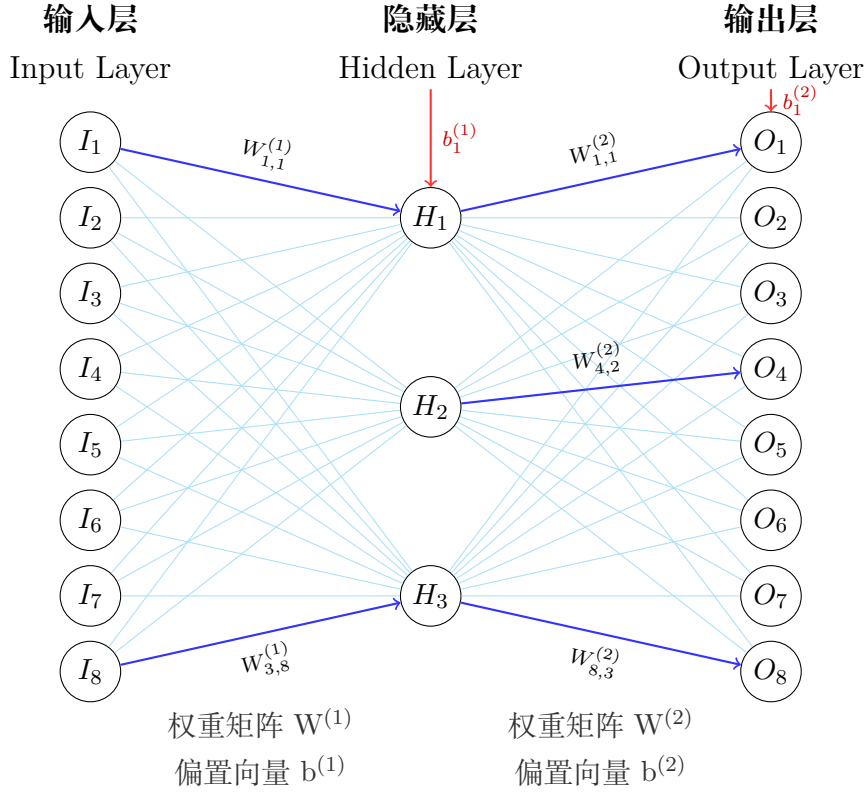


Figure 2: 带有权重与偏置标号的 8-3-8 自编码器神经网络结构图

针对本作业要求编解码 8×8 单位矩阵的 8-3-8 网络结构，我们将推导过程分为以下几个步骤。

2.1 1. 符号定义与网络结构

设网络输入为 x ，真实标签为 y ，网络预测输出为 o 。各层索引从 1 开始定义：

- **输入层**：8 个节点，记为 x_i ($i = 1, 2, \dots, 8$)。
- **隐藏层**：3 个节点，记为 h_j ($j = 1, 2, 3$)。输入层到隐藏层的权重矩阵记为 $W^{(1)}$ ，其中 $w_{ji}^{(1)}$ 表示从输入节点 i 到隐藏节点 j 的权重；偏置项记为 $b_j^{(1)}$ 。
- **输出层**：8 个节点，记为 o_k ($k = 1, 2, \dots, 8$)。隐藏层到输出层的权重矩阵记为 $W^{(2)}$ ，其中 $w_{kj}^{(2)}$ 表示从隐藏节点 j 到输出节点 k 的权重；偏置项记为 $b_k^{(2)}$ 。

网络规定激活函数为 Sigmoid 函数。Sigmoid 函数的一个极优美的数学性质是，其导数可以用自身表示，这为后续的推导带来了极大便利：

$$f(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

$$f'(z) = f(z)(1 - f(z)) \quad (2)$$

2.2 2. 前向传播 (Forward Propagation)

前向传播是数据从输入层流向输出层并产生预测值的过程。

首先，输入数据 x_i 经过权重加权求和并加上偏置，得到隐藏层的 ** 预激活值 ** $z_j^{(1)}$ ，再通过 Sigmoid 激活函数得到隐藏层的输出 h_j ：

$$z_j^{(1)} = \sum_{i=1}^8 w_{ji}^{(1)} x_i + b_j^{(1)} \quad (3)$$

$$h_j = f(z_j^{(1)}) \quad (4)$$

同理，隐藏层的数据流向输出层，计算出网络最终的预测值 o_k ：

$$z_k^{(2)} = \sum_{j=1}^3 w_{kj}^{(2)} h_j + b_k^{(2)} \quad (5)$$

$$o_k = f(z_k^{(2)}) \quad (6)$$

为了衡量网络的预测值 o_k 与真实标签 y_k 之间的差距，我们定义 ** 均方误差 (Mean Squared Error, MSE) ** 作为损失函数 L 。前面乘上 $\frac{1}{2}$ 是为了在求导时与平方项带来的系数 2 抵消，简化公式：

$$L = \frac{1}{2} \sum_{k=1}^8 (o_k - y_k)^2 \quad (7)$$

2.3 3. 反向传播 (Backward Propagation)

我们的最终目的是求损失函数 L 对所有权重 w 和偏置 b 的偏导数。为了推导清晰，我们引入 ** “误差项” δ ** 的概念，它表示损失函数对某一层预激活值 z 的偏导数。

2.3.1 3.1 输出层梯度推导

我们先看隐藏层到输出层的权重 $w_{kj}^{(2)}$ 。根据链式法则，损失 L 随 $w_{kj}^{(2)}$ 的变化可以分解为三步：

$$\frac{\partial L}{\partial w_{kj}^{(2)}} = \frac{\partial L}{\partial o_k} \cdot \frac{\partial o_k}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial w_{kj}^{(2)}} \quad (8)$$

我们先计算前两项的乘积，并将其定义为 ** 输出层误差项 $\delta_k^{(2)}$ **：

$$\begin{aligned} \delta_k^{(2)} &= \frac{\partial L}{\partial z_k^{(2)}} = \frac{\partial L}{\partial o_k} \cdot \frac{\partial o_k}{\partial z_k^{(2)}} \\ &= (o_k - y_k) \cdot f'(z_k^{(2)}) \\ &= (o_k - y_k) \cdot o_k(1 - o_k) \end{aligned} \quad (9)$$

而第三项，根据公式 (5) 可知 $\frac{\partial z_k^{(2)}}{\partial w_{kj}} = h_j$ 。将它们组合起来，并同理对偏置 $b_k^{(2)}$ 求导，得到输出层的最终梯度：

$$\frac{\partial L}{\partial w_{kj}^{(2)}} = \delta_k^{(2)} \cdot h_j \quad (10)$$

$$\frac{\partial L}{\partial b_k^{(2)}} = \delta_k^{(2)} \quad (11)$$

物理意义：输出层权重的调整量，等于该输出节点的误差 ($\delta_k^{(2)}$) 乘以与之相连的隐藏层输入 (h_j)。

2.3.2 3.2 隐藏层梯度推导

接下来我们推导输入层到隐藏层的权重 $w_{ji}^{(1)}$ 。这里最关键的难点在于：**隐藏层的某一个节点 j ，它的输出 h_j 会连接到下一层所有的 8 个输出节点上。因此，在计算隐藏层的误差时，必须把下一层所有 8 个节点传回来的误差累加起来。

我们定义 ** 隐藏层误差项 $\delta_j^{(1)}$ **：

$$\begin{aligned} \delta_j^{(1)} &= \frac{\partial L}{\partial z_j^{(1)}} = \left(\sum_{k=1}^8 \frac{\partial L}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial h_j} \right) \cdot \frac{\partial h_j}{\partial z_j^{(1)}} \\ &= \left(\sum_{k=1}^8 \delta_k^{(2)} \cdot w_{kj}^{(2)} \right) \cdot f'(z_j^{(1)}) \\ &= \left(\sum_{k=1}^8 \delta_k^{(2)} \cdot w_{kj}^{(2)} \right) \cdot h_j(1 - h_j) \end{aligned} \quad (12)$$

同理，根据公式 (3)，预激活值对权重的偏导为对应的输入 $\frac{\partial z_j^{(1)}}{\partial w_{ji}^{(1)}} = x_i$ 。得到隐藏层的最终梯度：

$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} \cdot x_i \quad (13)$$

$$\frac{\partial L}{\partial b_j^{(1)}} = \delta_j^{(1)} \quad (14)$$

物理意义：隐藏层接收到的总误差，等于下一层传回来的误差 ($\delta_k^{(2)}$) 根据权重大小 ($w_{kj}^{(2)}$) 按比例分配的总和，再乘以自身的激活函数梯度。

2.4 4. 梯度下降算法 (Gradient Descent)

在获得每一层参数的偏导数（即梯度）后，就相当于找到了误差在当前点上升最快的方向。为了让误差变小，我们需要让参数沿着梯度的 ** 反方向 ** 移动一小步。

引入超参数——** 学习率 (Learning Rate, η)**, 控制每次参数更新的步长。参数更新的规则如下:

更新输出层参数:

$$w_{kj}^{(2)} \leftarrow w_{kj}^{(2)} - \eta \frac{\partial L}{\partial w_{kj}^{(2)}} \quad (15)$$

$$b_k^{(2)} \leftarrow b_k^{(2)} - \eta \frac{\partial L}{\partial b_k^{(2)}} \quad (16)$$

更新隐藏层参数:

$$w_{ji}^{(1)} \leftarrow w_{ji}^{(1)} - \eta \frac{\partial L}{\partial w_{ji}^{(1)}} \quad (17)$$

$$b_j^{(1)} \leftarrow b_j^{(1)} - \eta \frac{\partial L}{\partial b_j^{(1)}} \quad (18)$$

通过不断地重复 “前向传播计算误差 $\rightarrow$ 反向传播计算梯度 $\rightarrow$ 梯度下降更新参数” 的过程, 网络最终能够学会编解码 8×8 单位矩阵的映射关系, 使得损失函数 L 趋近于 0。